



OpenSpec: Spec-Driven Development nell'era degli Agenti AI

Un nuovo paradigma per la collaborazione tra Umani e Intelligenza Artificiale

Matteo Baccan

DevFest Modena 2026 - 3/4 Ottobre 2026, Modena

Track: AI & Machine Intelligence



Google
Developer
Groups

Di cosa parleremo

01 **Dal Vibe Coding a OpenSpec**

Perché i prompt improvvisati non scalano e serve una struttura

03 **Il Linguaggio dell'Intento**

EARS + BDD: specifiche che l'agente non può fraintendere

05 **Dal Team al Legacy**

Adozione incrementale sul brownfield e anti-pattern da evitare

02 **La Meccanica di OpenSpec**

Directory, artefatti e Delta Specs: l'anatomia del framework

04 **Il Ciclo di Esecuzione**

Propose → Apply → Archive: la macchina a stati delle modifiche

06 **Perché Tutto Questo Conta**

Determinismo, intento persistente e il ruolo dell'Agentic Engineer



Google
Developer
Groups

01

Dal Vibe Coding a OpenSpec

Perché i prompt improvvisati non scalano e serve una struttura



Google
Developer
Groups

Il Problema: Agenti AI senza struttura

Gli agenti AI sono collaboratori attivi, ma senza struttura generano **vibe coding**: prompt informali, contesto volatile, drift dei requisiti.

- I requisiti vivono solo nella chat: nessuna documentazione persistente
- Allucinazioni architetturali e scelte inconsistenti
- Debito tecnico invisibile e difficilmente tracciabile



Google
Developer
Groups

La Soluzione: Spec-Driven Development

Lo **Spec-Driven Development** inverte il paradigma: **la struttura prima del codice**.

	Vibe Coding	Spec-Driven Development
Flusso	Prompt → Codice	Idea → Specifica → Architettura → Task → Codice
Artefatti	Nessuno (solo chat)	<code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs
Verificabilità	Impossibile	Ogni fase produce output controllabile



Google
Developer
Groups

Cos'è OpenSpec?

OpenSpec è un framework open-source progettato per la Spec-Driven Development.

Non richiede paywall o API key proprietarie per esistere come metodo, non è vincolato a un IDE specifico o a un singolo vendor AI.

È un sistema basato su file Markdown che funge da "controllo di versione per l'intento"

- **Zero lock-in**
- Specifiche portabili
- Valore che resta nel repository.



Google
Developer
Groups

OpenSpec: i tre pilastri

Pilastro 1: Brownfield-First ($1 \rightarrow n$)

Ottimizzato per l'evoluzione di codebase esistenti, non solo per prototipi greenfield ($0 \rightarrow 1$).

Permette di modificare logiche complesse in sicurezza, senza riscrivere tutto da zero.

Pilastro 2: Architettura Leggera

Markdown + Git come base operativa: niente infrastruttura pesante, niente database complessi.

Il metodo resta trasparente, versionabile e vicino al flusso reale del team.

Pilastro 3: Agnosticismo Totale

Zero lock-in verso IDE, modelli o vendor: il valore vive nelle specifiche, non nella piattaforma.

Se cambi agente o ambiente, il processo resta intatto perché l'intento rimane nel repository.



Google
Developer
Groups

La Filosofia di OpenSpec

L'idea centrale è che la specifica sia la "fonte della verità", non il codice.

I documenti fungono da istruzioni eseguibili e vincolanti per gli agenti AI, non solo come suggerimenti o linee guida.

Il risultato non nasce da un'interpretazione libera: con OpenSpec c'è **solo intento deterministico**, espresso in un contratto scritto che l'agente deve seguire.



Google
Developer
Groups

Living Documentation

Le specifiche sono mantenute in Git accanto al codice sorgente.

Non diventano mai obsolete perché si evolvono in parallelo al sistema, rendendo la documentazione un prodotto vivo dello sviluppo.



Google
Developer
Groups

OpenSpec vs Strumenti di Project Management

I ticket nei sistemi tradizionali (come Jira o Linear) sono ottimi per gli umani, ma difficili da consultare in tempo reale dagli agenti AI.

OpenSpec porta le specifiche direttamente nel repository, dove l'agente opera.



Google
Developer
Groups

Ecosistema agnostico: zero lock-in come scelta

Nodo centrale	Ecosistema collegabile
OpenSpec come livello stabile di intento e governance nel repository	Kilo Code, Cursor, Claude Code, Windsurf, GitHub Copilot e oltre 30 strumenti compatibili

Capacità	Impatto operativo
Integrazioni intensive	Uso tramite CLI e slash commands nativi nei principali IDE/LLM, senza riscrivere le specifiche
Supporto universale (AGENTS.md)	Le istruzioni restano leggibili da assistenti diversi (anche futuri), preservando storico e coerenza del processo. Sta diventando lo standard de facto: da Claude Code 2.1.277, in mancanza di CLAUDE.md viene letto AGENTS.md

Risultato: cambi strumento quando vuoi, senza perdere memoria progettuale né controllo sull'intento.



Google
Developer
Groups

Approccio "Brownfield-First"

Molti framework AI sono ottimizzati per progetti nuovi (greenfield, $0 \rightarrow 1$).

OpenSpec brilla nei progetti esistenti (brownfield, $1 \rightarrow n$), dove l'integrazione di nuove funzionalità senza rompere le vecchie è cruciale.



Google
Developer
Groups

02

La Meccanica di OpenSpec

Directory, artefatti e Delta Specs:
l'anatomia del framework



Google
Developer
Groups

L'Architettura del Framework

Tutto risiede in una singola directory radice all'interno del progetto:

openspec

Questa directory agisce come la memoria a lungo termine dell'agente AI.



Google
Developer
Groups

La Separazione dello Stato

Il framework separa fisicamente:

- La descrizione dello stato attuale del sistema.
- Le proposte per le modifiche future.

Questa distinzione garantisce transizioni sicure e modifiche parallele.



Google
Developer
Groups

L'architettura a due cartelle: specs/

L'Essere: la fonte della verità.

Documentazione del comportamento attuale del sistema, organizzata per domini logici.
L'agente la consulta prima di proporre o scrivere codice.

- specs/auth/spec.md
- specs/payments/spec.md
- specs/ui/spec.md



Google
Developer
Groups

L'architettura a due cartelle: `changes /`

Il Divenire: il workspace di modifica.

Laboratorio isolato per ogni nuova feature o bug fix. Nessun conflitto con le specifiche principali finché non si archivia.

Più sviluppatori (e più agenti AI) possono preparare funzionalità diverse simultaneamente, senza inquinare la logica base.

- `changes/add-oauth-login/`
 - `proposal.md`
 - `design.md`
 - `tasks.md`
 - `specs/...` (Delta)



Google
Developer
Groups

Configurazione: Il File `config.yaml`

Funziona come la "costituzione" tecnica del progetto.

Sostituisce configurazioni frammentate fornendo un set di regole chiare, stack tecnologici e convenzioni di sviluppo.



Google
Developer
Groups

Governance attiva: come `config.yaml` guida ogni richiesta

<code>config.yaml</code>	Funzione	Effetto sull'agente
<code>schema</code>	Definisce workflow e struttura degli artefatti attesi	L'agente non improvvisa formati: produce output conformi
<code>context</code>	Fornisce architettura globale, stack, standard tecnici	Ogni piano parte dagli stessi vincoli di progetto
<code>rules</code>	Impone guardrail granulari per fase e artefatto	Riduce deviazioni, allucinazioni e scelte fuori policy

Risultato operativo: il file non resta passivo nel repository, viene iniettato nel prompt di pianificazione e rende la conformità tecnica sistematica.



Google
Developer
Groups

Gli Artefatti della Pianificazione

Una cartella di modifica genera sempre un set standard di artefatti Markdown:

- `proposal.md`
- `design.md`
- `tasks.md`
- Le Delta Specs



Google
Developer
Groups

Anatomia end-to-end di una proposta

Fase	File	Domanda chiave	Ruolo operativo
1	proposal.md	Perché / Cosa stiamo cambiando?	Definisce intento strategico e ambito lavori (business case iniziale).
2	design.md	Come lo realizziamo?	Fissa decisioni tecniche: architettura, flussi dati, librerie e vincoli.
3	tasks.md	Come eseguiamo in modo verificabile?	Scompone in task atomici e sequenziali, eseguibili dall'agente senza ambiguità.
4	Delta Specs	Cosa cambia nella Source of Truth?	Applica patch ai requisiti con sezioni ADDED , MODIFIED , REMOVED .

Flusso completo: proposal.md → design.md → tasks.md → Delta → consolidamento → specs/



Google
Developer
Groups

L'Artefatto 1: `proposa1.md`

Cattura l'intento strategico. Risponde alle domande "perché stiamo facendo questa modifica?" e "qual è lo scopo principale?".

È il punto di allineamento iniziale tra umano e macchina.

`## Why`

Gli utenti abbandonano la registrazione: serve il login social.

`## What Changes`

- Aggiunta autenticazione OAuth (Google) al flusso di login
- Nessun impatto sul login email/password esistente



Google
Developer
Groups

L'Artefatto 2: `design.md`

È l'ancora tecnica. Delinea scelte di database, flussi di dati e specifiche librerie da utilizzare. Impedisce all'agente di deviare dai pattern stabiliti "immaginando" soluzioni creative ma errate.

Decisioni tecniche

- Libreria: passport con strategy passport-google-oauth20
- Nel DB solo l'id utente Google: nessun token persistito
- Sessioni: riuso del meccanismo esistente (Redis)



Google
Developer
Groups

L'Artefatto 3: `tasks.md`

Una checklist numerata di azioni da compiere.

Funge da registro di avanzamento. L'agente aggiorna le spunte in tempo reale mentre scrive il codice, garantendo totale trasparenza.

1. Backend

- [x] 1.1 Aggiungere endpoint GET /auth/google
- [x] 1.2 Gestire il callback e creare l'utente
- [] 1.3 Test di integrazione del flusso completo



Google
Developer
Groups

L'Importanza dei Task Atomici

I task in `tasks.md` devono essere sufficientemente piccoli da essere implementati dall'agente in autonomia, senza richiedere costanti chiarimenti o interventi dell'utente.



Google
Developer
Groups

L'Artefatto 4: Delta Specs

La vera innovazione di OpenSpec. Invece di riscrivere l'intera specifica di sistema, l'agente crea una specifica "differenziale" (Delta) che mostra solo cosa cambierà.



Google
Developer
Groups

Perché usare le Delta Specs?

In codebase enormi, rigenerare tutta la documentazione consumerebbe troppi token e tempo.

Le Delta Specs si focalizzano solo sull'incremento funzionale, ottimizzando i costi e mantenendo alta l'accuratezza.



Google
Developer
Groups

Anatomia di una Delta Spec

Le Delta Specs utilizzano intestazioni chiare per istruire il processo di fusione futuro:

- ADDED Requirements
- MODIFIED Requirements
- REMOVED Requirements



Google
Developer
Groups

Delta Tags: ADDED

Definisce comportamenti completamente nuovi.

Esempio: L'aggiunta di un sistema di autenticazione a due fattori in un progetto che prima aveva solo login base.

ADDED Requirements

Requirement: Two-Factor Authentication

The system SHALL richiedere un secondo fattore al login.

Scenario: Login con 2FA attiva

- WHEN un utente con 2FA invia credenziali valide
- THEN viene richiesto un codice OTP prima di creare la sessione



Google
Developer
Groups

Delta Tags: MODIFIED

Descrive l'alterazione di comportamenti esistenti.

Richiede di riscrivere l'intero requisito aggiornato, garantendo che nessuna regola pregressa venga persa per distrazione dell'agente.

MODIFIED Requirements

Requirement: Session Duration

The system SHALL scadere la sessione dopo 30 minuti di inattività (prima: 24 ore), mantenendo il "ricordami" per i dispositivi fidati.



Google
Developer
Groups

Delta Tags: REMOVED

Segnala funzionalità deprecate o da rimuovere.

Garantisce che il codice morto venga eliminato e che i vecchi test associati vengano disattivati correttamente.

```
## REMOVED Requirements
```

```
### Requirement: Legacy XML Export
```

```
**Reason**: sostituito dall'export JSON
```

```
**Migration**: i client devono passare a /api/export/json
```



Google
Developer
Groups

03

Il Linguaggio dell'Intento

EARS + BDD: specifiche che l'agente non può fraintendere



Google
Developer
Groups

Scrivere Specifiche Efficaci

Le specifiche non devono essere istruzioni di programmazione step-by-step.

Devono descrivere il comportamento osservabile del sistema dall'esterno. Sono contratti di business, non tutorial di codice.



Google
Developer
Groups

Il linguaggio dell'intento: EARS + BDD

Asse	EARS (obbligazione)	BDD (esecuzione)
Scopo	Ridurre ambiguità nei requisiti	Rendere verificabili i comportamenti
Forma	Parole chiave normative: SHALL , MUST , SHOULD	Struttura scenario: GIVEN , WHEN , THEN
Domanda a cui risponde	"Cosa è obbligatorio?"	"Come si osserva il risultato?"
Esito pratico	Contratti chiari per l'agente	Test di accettazione derivabili

Regola operativa: prima definiamo il vincolo con EARS, poi ne proviamo l'esecuzione con BDD.



Google
Developer
Groups

Sintassi EARS

OpenSpec adotta l'approccio EARS (Easy Approach to Requirements Syntax).

L'obiettivo è minimizzare l'ambiguità utilizzando parole chiave vincolanti per definire i livelli di obbligazione.

Non nasce nel software. EARS viene da Rolls-Royce: Alistair Mavin e colleghi la ricavarono analizzando le normative di aeronavigabilità del sistema di controllo di un motore a reazione, e la pubblicarono a IEEE RE'09 nel 2009.

Oggi è adottata da Airbus, Bosch, Honeywell, Intel, NASA e Siemens: la stessa sintassi che tiene in volo un motore ora vincola un agente AI.



Google
Developer
Groups

Le parole chiave: SHALL e MUST

Rappresentano comportamenti obbligatori e non negoziabili.

Se un test fallisce su un requisito **SHALL**, l'agente sa che l'implementazione è categoricamente errata.



Google
Developer
Groups

La parola chiave: **SHOULD**

Indica una raccomandazione o un comportamento desiderato, ma con margini di flessibilità.

Permette all'agente di adattarsi a vincoli tecnici imprevisti durante la scrittura del codice.



Google
Developer
Groups

Scenari Verificabili

Ogni requisito deve essere accompagnato da scenari pratici derivati dal Behavior-Driven Development (BDD).

Forniscono all'agente esempi inequivocabili di successo e fallimento.



Google
Developer
Groups

Il Modello **GIVEN / WHEN / THEN**

- **GIVEN:** Il contesto iniziale (es. "Dato un utente non autenticato").
- **WHEN:** L'azione scatenante (es. "Quando visita la dashboard").
- **THEN:** Il risultato atteso (es. "Allora viene reindirizzato al login").



Google
Developer
Groups

Dalla Specifica ai Test

Con una struttura GIVEN/WHEN/THEN chiara, l'agente AI è in grado di generare automaticamente i test unitari o e2e corrispondenti, chiudendo il ciclo della qualità del software.



Google
Developer
Groups

04

Il Ciclo di Esecuzione

Propose → Apply → Archive: la macchina a stati delle modifiche



Google
Developer
Groups

Il Ciclo Operativo (Workflow)

OpenSpec definisce una macchina a stati immutabile a tre fasi per ogni modifica:

- Propose: `/opsx:propose`
- Apply: `/opsx:apply`
- Archive: `/opsx:archive`

Ogni transizione ha un output verificabile e impedisce di passare alla fase successiva senza allineamento.

Fase zero opzionale: `/opsx:explore`, un "thinking partner" senza vincoli che legge il codice e pesa le alternative prima di formalizzare la proposta.

Il profilo esteso aggiunge comandi come `/opsx:verify`, `/opsx:ff`, `/opsx:continue` e `/opsx:onboard`.



Google
Developer
Groups

La macchina a stati di OpenSpec

Fase	Obiettivo	Output della fase	Gate di passaggio
Propose	Allineare intento e ambito prima del codice	Cartella in <code>changes/</code> con <code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs	Revisione umana dell'intento
Apply	Implementare senza deviazioni dai vincoli	Codice + test aderenti a <code>design.md</code> e <code>tasks.md</code>	Verifica qualità su MUST/SHALL
Archive	Consolidare la modifica nella verità di sistema	Fusione Delta Specs in <code>specs/</code> + archivio storico della change	Stato aggiornato e audit trail persistente

Logica del ciclo: cattura presto il disallineamento (Propose), esegui in modo deterministico (Apply), consolida nella Source of Truth (Archive).



Google
Developer
Groups

Fase 1: Proposta (Propose)

L'agente non scrive codice. Analizza la richiesta dell'umano e genera la cartella in `changes/` con la proposta, i task e le Delta Specs.



Google
Developer
Groups

La Revisione dell'Intento

Questo è il momento chiave per lo sviluppatore umano.

Modificare un file Markdown errato richiede pochi secondi. Correggere un'architettura software errata dopo che è stata scritta richiede giorni.



Google
Developer
Groups

La Scissione Cognitiva dell'Agente

Perché separare proposta e implementazione aumenta la qualità del codice generato?

Nel vibe coding il modello divide il proprio **budget di attenzione** tra tre compiti contemporaneamente:

- Risolvere il problema di business
- Progettare l'architettura software
- Scrivere codice sintatticamente corretto

Risultato: precisione ridotta su tutti e tre i fronti, più iterazioni, più errori architetturali.



Google
Developer
Groups

Focalizzazione: il 100% su un compito alla volta

Con OpenSpec ogni fase riceve l'intera capacità computazionale del modello:

Fase	Focus dell'agente
Propose	100% sull'architettura e le decisioni di design
Apply	100% sulla correttezza e completezza del codice
Archive	100% sulla coerenza documentale e la Source of Truth

Più focus su un compito → Meno errori → Meno cicli di correzione



Google
Developer
Groups

Fase 2: Implementazione (Apply)

Solo quando gli artefatti sono approvati, l'agente inizia l'implementazione vera e propria.

Legge i compiti in `tasks.md` e produce il codice rigorosamente entro i confini stabiliti nel `design.md`.



Google
Developer
Groups

L'Esecuzione dell'Agente

Durante l'Apply, l'agente AI si trasforma in un mero esecutore.

Smette di indovinare le intenzioni e si limita a tradurre specifiche perfette in codice funzionante.



Google
Developer
Groups

Gestione della Qualità

Prima di considerare chiusa la fase di implementazione, l'agente verifica il codice prodotto rispetto alle specifiche stabilite, correggendo autonomamente eventuali deviazioni dai requisiti MUST/SHALL.



Google
Developer
Groups

Fase 3: Archiviazione (Archive)

Una volta che il codice funziona ed è stato testato, la modifica viene conclusa.

Questo è il momento in cui l'intento temporaneo diventa documentazione permanente.



Google
Developer
Groups

La Fusione della Verità

Il sistema unisce automaticamente le Delta Specs nei file originali della cartella `specs/`.

Il sistema si aggiorna: la nuova funzionalità fa ora ufficialmente parte della "Source of Truth" del progetto.



Google
Developer
Groups

L'Archivio Storico

La cartella originale della modifica viene spostata in un archivio storico (es. ordinato per data).

Si preserva così l'audit trail delle decisioni architetturali per i futuri sviluppatori.



Google
Developer
Groups

Funzionalità Avanzate: Schemi Personalizzati

Team diversi hanno processi diversi. OpenSpec permette di definire schemi personalizzati per generare artefatti specifici (es. audit di sicurezza obbligatori) prima dell'implementazione.



Google
Developer
Groups

Parallelismo e sincronizzazione

OpenSpec scala su tre assi:

Orizzontale: branch paralleli

Più agenti su feature diverse, ognuno con la propria cartella **changes/**; il ciclo **Propose → Apply → Archive** guida il merge ordinato nella Source of Truth.

Verticale: Project Management

Via MCP dialoga con Linear/Jira: il backlog resta allineato allo stato reale del codice.

Cross-repo: Stores (beta)

La pianificazione vive in un repository dedicato e condiviso; i repo di codice si allineano dopo.

Messaggio chiave: OpenSpec scala in orizzontale, in verticale e tra repository



Google
Developer
Groups

Orchestrazione Multi-Agente

OpenSpec non è solo un'interfaccia tra umano e LLM.

È un livello di astrazione che permette a **team di agenti specializzati** di collaborare autonomamente su sistemi complessi.

Invece di affidarsi a un singolo agente generico, il supervisore umano coordina ruoli distinti.



Google
Developer
Groups

I Ruoli nel Sistema Multi-Agente

Agente Architetto

Consuma `config.yaml` e produce Record di Decisione Architetture (ADR).

Genera la struttura iniziale OpenSpec e pianifica la suddivisione logica del sistema.

Agente Orchestratore

Legge il `tasks.md` prodotto dall'architetto e delega il lavoro ai team specializzati.

Supervisiona l'avanzamento senza scrivere codice direttamente.

Team di Sviluppo AI

Agenti iperspecializzati per dominio: Data Access, Business Logic, Frontend.

Lavorano in parallelo sui propri task atomici, vincolati alle specifiche.

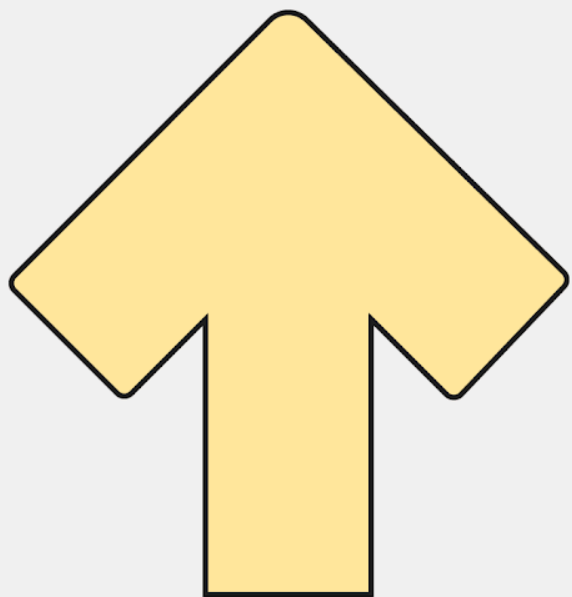


Google
Developer
Groups

05

Dal Team al Legacy

Adozione incrementale sul brownfield
e anti-pattern da evitare



Google
Developer
Groups

Il Problema del Codice Legacy

Come si introduce OpenSpec in un progetto esistente di 100.000 righe di codice senza documentazione?

Non riscrivendo tutto da zero.



Google
Developer
Groups

Brownfield adozione incrementale

La scala dell'adozione	<code>/opsx:onboard</code> (reverse engineering)
1. Non fermare il mondo Nessun bisogno di produrre specifiche upfront per milioni di righe.	Comando dedicato per accelerare la partenza su moduli complessi o poco documentati.
2. Sviluppo just-in-time Usa OpenSpec sulla prossima feature o sul bug critico, dove stai già intervenendo.	L'agente analizza il codice esistente per estrarre le regole di business già implementate.
3. Accumulo organico La Source of Truth cresce naturalmente ad ogni ciclo <code>Propose -> Apply -> Archive</code> .	Generazione assistita di file <code>spec.md</code> retroattivi, da rifinire e validare con il team.

Non serve documentare tutto prima di iniziare.



Google
Developer
Groups

Come si parte

Servono Node.js **20.19.0** o superiore e un package manager supportato.

```
npm install -g @fission-ai/openspec@latest  
openspec --version  
  
cd your-project  
openspec init
```

openspec init genera la struttura nel progetto esistente: da lì il primo ciclo **Propose -> Apply -> Archive** parte sulla prossima feature.



Google
Developer
Groups

Memoria Episodica per il Codice Legacy

In un contesto brownfield un singolo prompt iniziale non basta.

OpenSpec supporta un'architettura a **memoria episodica** che trasforma i fallimenti in istruzioni correttive persistenti:

Componente	Funzione
Agente Riflettore	Si attiva ogni N scambi, analizza la cronologia recente, identifica errori architetturali e produce una root-cause analysis
Agente Curatore	Legge l'analisi del Riflettore e la traduce in mutazioni di regole (aggiunte, aggiornamenti, rimozioni)
Playbook Eseguitibile	Regole salvate in un file JSON di sessione: l'agente costruisce il proprio manuale tattico mentre lavora

L'agente impara dai propri errori e non li ripete nella stessa sessione.



Google
Developer
Groups

Mitigazione dei rischi: 4 anti-pattern da evitare

Il cimitero delle proposte

Problema: dimenticare `Archive` e lasciare change mai consolidate.

Fix: integrare l'archiviazione nel workflow operativo e nei controlli di qualità.

Il micro-management dell'AI

Problema: scrivere dettagli implementativi nelle specifiche principali.

Fix: tenere il comportamento nelle spec e spostare il "come" in `design.md`.

Burocrazia per modifiche banali

Problema: usare una SDD completa per un cambio minimo e non rischioso.

Fix: applicare progressive rigor e mantenere un approccio leggero.

Ignorare la formattazione Delta

Problema: non usare `ADDED`, `MODIFIED`, `REMOVED` in modo strutturato.

Fix: rispettare i metadati Delta per merge puliti e consolidamento affidabile.



Google
Developer
Groups

06

Perché Tutto Questo Conta

Determinismo, intento persistente e
il ruolo dell'Agentic Engineer



Google
Developer
Groups

Impatto strategico

Determinismo + intento persistente = scalabilità umano-AI

Precisione	Resilienza	Evoluzione del ruolo
Specifiche eseguibili: più accuratezza al primo tentativo, meno rilavorazioni.	La conoscenza architetture non muore nella chat: resta al team e resiste al turnover.	Il valore umano passa dallo "scrivere tutto" al garantire che il codice sia quello necessario.

Sintesi: quando l'implementazione è deterministica e l'intento rimane persistente, il sistema scala senza perdere qualità.



Google
Developer
Groups

Il nuovo professionista: l'Agentic Engineer

Lo SDD forma una nuova categoria professionale che va oltre il programmatore tradizionale.

Un **ingegnere di sistemi agentici** che padroneggia:

- La progettazione del contesto per gli agenti
- La segmentazione di carichi di lavoro complessi in task atomici
- La prevenzione della corruzione della memoria episodica
- Il coordinamento tra molteplici collaboratori sintetici specializzati
- La manutenibilità a lungo termine delle infrastrutture software

Il codice è l'output. La specifica è la competenza.



Google
Developer
Groups

Conclusione

OpenSpec non riguarda lo scrivere meno codice, ma garantire che il codice generato dalle intelligenze artificiali sia **esattamente quello necessario**.

L'arte di definire l'intento è la competenza più preziosa del futuro.



Google
Developer
Groups



AMA

Ask Me Anything: chiedimi quello che vuoi.

Le mucche dormono in piedi?

Quante uova fa una gallina in un anno?



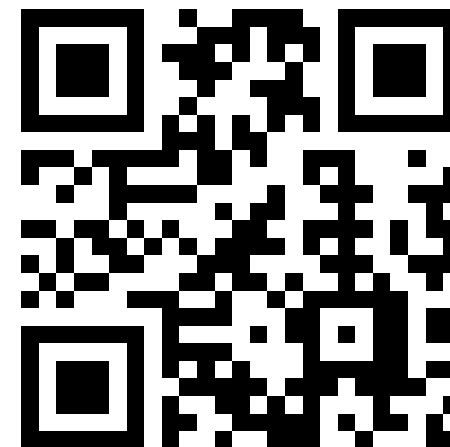
Google
Developer
Groups

Contatti



Matteo Baccan

Rivoluzione Digitale
head of development



<https://www.baccan.it>

"Smetti di chattare, inizia a governare."



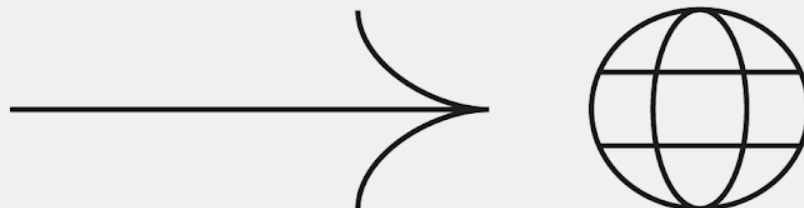
Google
Developer
Groups

Chi devo ringraziare per queste slide?

- DevFest Modena: per avermi invitato a parlare qui per la prima volta
- Anthropic: per l'abbonamento Claude Code Max, regalato per i miei contributi al mondo open source
- Claude: per aver convertito il template PowerPoint in un template Marp in markdown
- Nano Banana Pro: per le immagini
- NotebookLM: per la prima scaletta e i riassunti dei podcast e video
- VSCode: per gestire il progetto GitHub
- Marp: per la presentazione



Google
Developer
Groups



2026



Google
Developer
Groups